



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



SISTEMAS DISTRIBUIDOS

Práctica 8. Microservicios

ALUMNO:

CRUZ SANTOS CRISTIAN OZIEL

PROFESOR:

CHADWICK CARRETO ARELLANO

GRUPO:

7CM5

FECHA: 14/05/2026

1. Antecedentes

Arquitectura de Microservicios

La arquitectura de microservicios es un enfoque de diseño de software donde una aplicación se descompone en un conjunto de servicios pequeños, independientes y débilmente acoplados. A diferencia de las arquitecturas monolíticas, cada microservicio se especializa en una tarea de negocio específica y se comunica a través de protocolos ligeros como HTTP/REST. Esto permite que el sistema sea más fácil de mantener, probar y escalar de forma independiente.

API Gateway y Enrutamiento de Peticiones

En un ecosistema de microservicios, el API Gateway actúa como un punto de entrada único para todos los clientes (en este caso, la PWA). Su función principal es el "Reverse Proxy" y el enrutamiento por contenido: recibe las peticiones, inspecciona la carga útil (JSON) y las redirige al microservicio correspondiente. Esto oculta la complejidad interna de la red al usuario final y centraliza tareas como la seguridad y el balanceo de carga.

2. Problemática

Aislamiento de Fallos y Escalabilidad Selectiva Incluso con una PWA funcional y un servidor web centralizado, existen riesgos operativos en sistemas de alta disponibilidad:

1. **Punto Único de Falla:** En un monolito, un error crítico en la lógica de las pizzas puede detener el procesamiento de hamburguesas o tacos, colapsando todo el restaurante.
2. **Escalabilidad Ineficiente:** Si la demanda de pizzas aumenta, en un monolito se debe replicar todo el servidor, desperdiciando recursos en funciones que no tienen demanda.
3. **Complejidad en la Gestión de Recursos:** La coordinación de semáforos para diferentes tipos de cocción en un solo proceso se vuelve difícil de gestionar a medida que crecen las opciones del menú.

3. Propuesta de Solución

Ecosistema Distribuido con API Gateway

Se propone la segmentación del sistema en tres entidades autónomas que cooperan entre sí:

- **Microservicios Especializados:** Se desplegaron dos nodos de cocina independientes (Pizzería en puerto 5001 y Cocina General en puerto 5002), cada uno con sus propios recursos y semáforos de control.

- **API Gateway Central:** Un servidor intermedio (puerto 5000) que sirve la PWA y gestiona el flujo de mensajes. Este componente mapea los IDs de los pedidos para saber a qué microservicio consultar el estatus, garantizando transparencia total para el mesero.
- **Tolerancia a Fallos por Aislamiento:** La arquitectura permite que cada microservicio opere en su propio hilo de ejecución y puerto, asegurando que el fallo de un nodo especializado no afecte la disponibilidad global del sistema.

4. Implementación

Los códigos desarrollados se encuentran en el siguiente repositorio de Github:

https://github.com/CruzCris/Sistemas_Distribuidos_7CM1

5. Resultados

Escenario de Prueba:

Se levantó la infraestructura completa (Gateway y dos Microservicios). Se accedió desde la PWA y se realizaron pedidos simultáneos de "Pizza Especial", "Hamburguesa Triple" y "Tacos al Pastor" para validar el enrutamiento.

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P8> py pizza_service.py
* Serving Flask app 'pizza_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
```

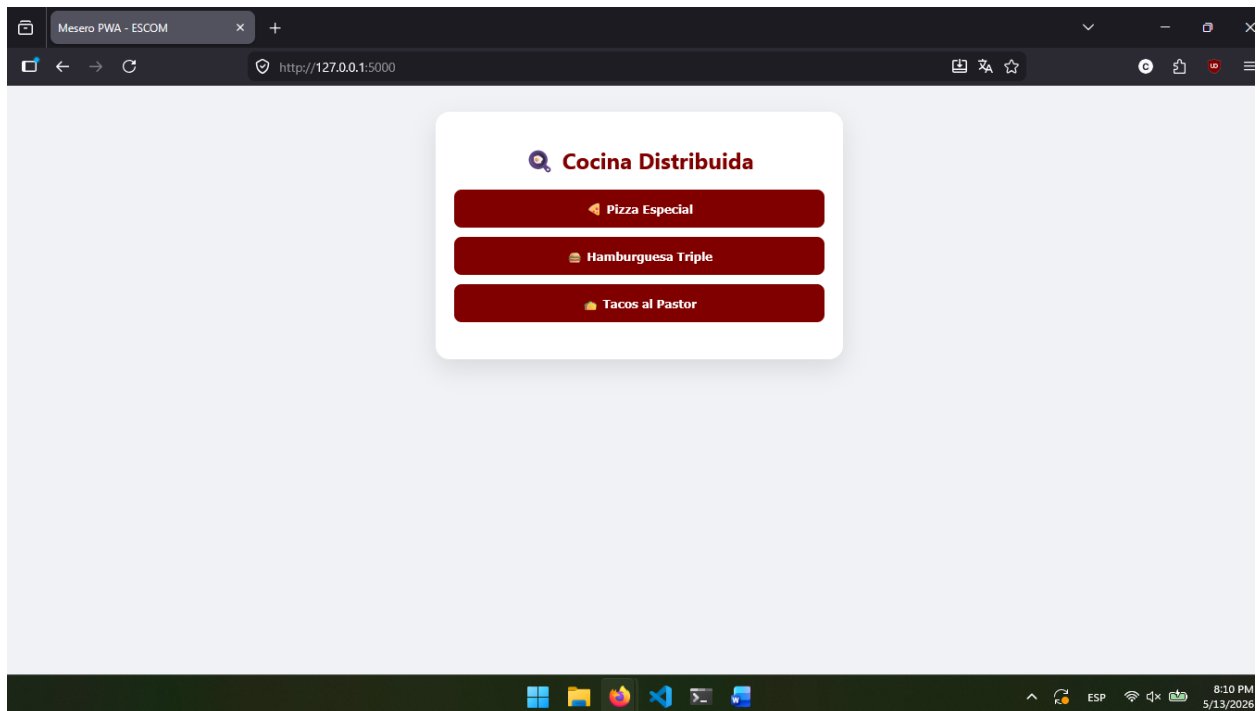
Levantamiento del servicio para las pizzas

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P8> py general_service.py
* Serving Flask app 'general_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production de
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

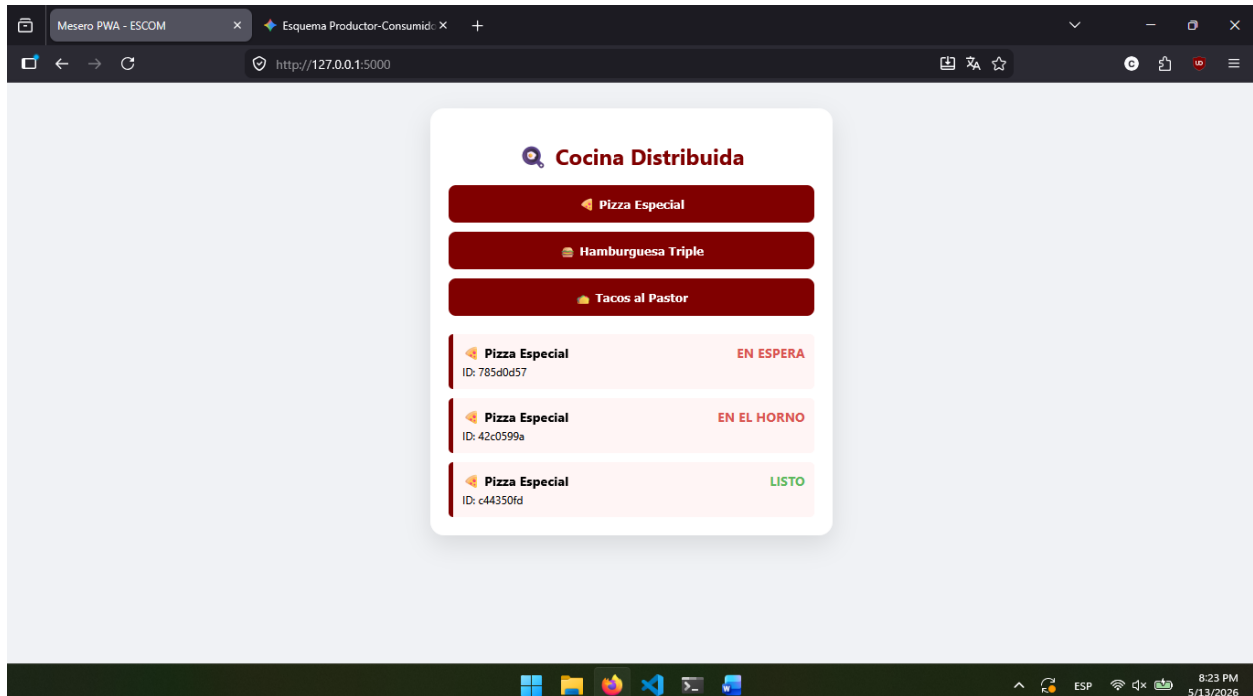
Levantamiento del servicio para los pedidos distintos a las pizzas

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P8> py gateway.py
* Serving Flask app 'gateway'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.100.20:5000
Press CTRL+C to quit
```

Levantamiento del gateway



Accedemos a la PWA construida en la práctica anterior



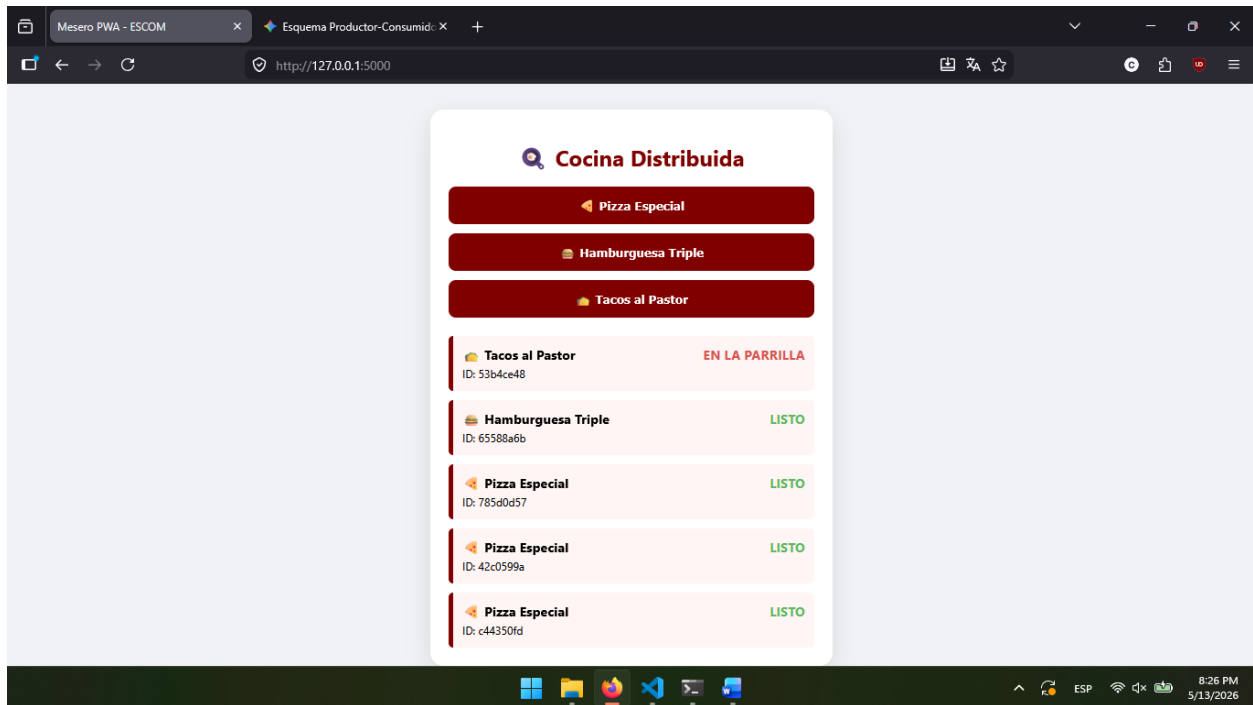
Levantamiento de 3 pedidos de pizza

```
127.0.0.1 - - [13/May/2026 20:23:22] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:24] "GET /status/c44350fd HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:29] "GET /status/c44350fd HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:34] "GET /status/c44350fd HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:42] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:44] "GET /status/42c0599a HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:49] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:49] "GET /status/42c0599a HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:51] "GET /status/785d0d57 HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:55] "GET /status/42c0599a HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:56] "GET /status/785d0d57 HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:24:01] "GET /status/785d0d57 HTTP/1.1" 200 -
```

Peticiones en el servicio de pizzas

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P8> py general_service.py
* Serving Flask app 'general_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

No hay peticiones en el servicio de cocina general aún



Levantamiento de 2 pedidos hacia la cocina general

```
Press CTRL+C to quit
127.0.0.1 - - [13/May/2026 20:25:55] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:25:56] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:25:57] "GET /status/65588a6b HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:25:58] "GET /status/53b4ce48 HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:26:02] "GET /status/65588a6b HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:26:03] "GET /status/53b4ce48 HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:26:07] "GET /status/65588a6b HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:26:08] "GET /status/53b4ce48 HTTP/1.1" 200 -
```

Peticiones en el servicio de la cocina general

```
Press CTRL+C to quit
127.0.0.1 - - [13/May/2026 20:23:22] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:24] "GET /status/c44350fd HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:29] "GET /status/c44350fd HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:34] "GET /status/c44350fd HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:42] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:44] "GET /status/42c0599a HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:49] "POST /cocinar HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:49] "GET /status/42c0599a HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:51] "GET /status/785d0d57 HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:55] "GET /status/42c0599a HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:23:56] "GET /status/785d0d57 HTTP/1.1" 200 -
127.0.0.1 - - [13/May/2026 20:24:01] "GET /status/785d0d57 HTTP/1.1" 200 -
```

El servicio de pizzas se mantuvo igual que antes

6. Conclusión

La transición a una arquitectura de **Microservicios** representa la culminación técnica de un sistema distribuido robusto. Se concluye que la división de responsabilidades no solo mejora la organización del código, sino que dota al sistema de una **resiliencia** que las arquitecturas anteriores (sockets y monolitos web) no poseían. El uso del **API Gateway** demostró ser una pieza fundamental para mantener la simplicidad en el cliente (PWA) mientras se escala la complejidad en el backend. Esta práctica valida los principios de la computación distribuida moderna: especialización, desacoplamiento y alta disponibilidad.